

O algoritmo de Otimização de baleias (WOA) - explorando um modelo metaheurístico de otimização

The whale optimization algorithm (WOA) - Exploring a metaheuristic optimization model

Fernando Amaru Romeu Alves [Universidade Federal de São João Del Rei]

Gabriel Leite Silva [Universidade Federal de São João Del Rei]

Ícaro Gomes Cook [Universidade Federal de São João Del Rei]

João Emanuel Pereira do Nascimento [Universidade Federal de São João Del Rei]

Resumo. Este trabalho visa explorar o algoritmo de otimização de baleias, um algoritmo meta-heurístico proposto em 2016 por Mirjalili [2016], inspirado no comportamento de caça de baleias-jubarte. Para isso, o algoritmo foi implementado de forma a resolver o clássico problema do caixeiro viajante, tendo sua performance comparada à do algoritmo genético, outro algoritmo meta-heurístico comum.

Abstract. This work aims to explore the Whale Optimization Algorithm, a meta-heuristic algorithm proposed in 2016 by Mirjalili [2016], inspired by the hunting behaviour of humpback whales. To achieve this, the algorithm was implemented in order to solve the travelling salesman problem, with its performance compared to that of the Genetic Algorithm, another commonly used meta-heuristic algorithm –

Palavras-chave: Inteligência artificial, Algoritmos de otimização, Algoritmo de otimização de baleias, Algoritmo Genético

Keywords: Artificial Intelligence, Optimization algorithm, Whale Optimization Algorithm, Genetic Algorithm

1 Introdução

Dentro de todas as contribuições acadêmicas proporcionadas pela Ciência da Computação, suas principais áreas de pesquisa envolvem a automação de solução de problemas, o auxílio na tomada de decisões, entre outras. Atualmente, com o surgimento de produtos embarcados com Inteligência Artificial Generativa tais como ChatGPT, Deepseek, Claude, etc, profissionais de diversas áreas. Em relação ao programador e ao desenvolvedor, eles devem possuir pleno domínio sobre as tecnologias desenvolvidas pela Ciência da Computação, de forma que consigam desenvolver sistemas robustos, eficientes e competitivos, e identificar pontos críticos que podem representar quedas de otimização.

O campo da Inteligência artificial é definido por Luger [2013] como "O ramo da ciência da computação que se ocupa com a automação do comportamento inteligente". Dentro dessa definição, duas questões fundamentais se levantam, sendo elas "O que é um comportamento inteligente?" e "como automatizá-lo?". Essas duas questões não possuem uma resposta direta, muito menos uma única resposta que seja indiscutivelmente a correta.

Uma abordagem nesse sentido é o estudo do processo de tomada de decisão, um processo comum e recorrente, que precisa ser realizado de forma lógica e racional, segundo Talbi [2009], este processo consiste nos seguintes passos:

- Formular o problema: Identificar o problema, destacando fatores internos e externos.
- Modelar o problema: Criar um modelo matemático abstrato que represente o problema, esse passo pode ser impreciso e gerar aproximações da realidade, ao invés da realidade em si.

- Otimizar o problema: Uma vez modelado, este processo gera uma "boa solução" para o problema, que pode ser a ideal ou abaixo da ideal. Como esse processo é feito através de um modelo abstrato ao invés do problema real, sua performance é dependente da acurácia do modelo em representar o problema real.
- Implementar a solução: A solução obtida é testada na prática e é implementada se for considerada "aceitável". Caso contrário, o processo de modelagem ou de otimização devem ser repetidos.

De acordo com Bastos [2025], "a medida em que a complexidade aumenta, o processo de tomada de decisão passa a ser um procedimento difícil e multifacetado", portanto, é necessário o desenvolvimento de técnicas e estratégias capazes de avaliar e compreender os diversos aspectos de uma situação, e, a partir disso, chegar a uma conclusão que seja a melhor decisão para cada momento.

Segundo Talbi [2009], os algoritmos meta-heurísticos são um grupo de algoritmos de otimização aproximada que "tem tido um grande aumento em popularidade nas últimas décadas por serem capazes de fornecer soluções aceitáveis em um tempo viável", o que pôde ser verificado por Bastos [2025], ao avaliar pesquisas explorando sua utilização em diversas áreas, tais como "a seleção de fornecedores, a gestão de projetos, a logística de transporte intermodal, a manufatura, o marketing, os problemas de roteamento de localização e a tomada de decisão em grupo", entre outras.

Ainda segundo Talbi [2009] Ao contrário de algoritmos exatos, que garantem encontrar soluções ótimas a qualquer custo para problemas específicos, algoritmos meta-heurísticos visam obter soluções aceitáveis sob custos aceitáveis e para uma vasta gama de problemas. Isso significa que a adoção de

um algoritmo meta-heurísticos pode ser benéfico para situações em que se busca minimizar o tempo de implementação ou execução de uma solução, mesmo que isso possa implicar em algum custo de performance.

O objetivo deste trabalho é explorar a capacidade de otimização desse modelo de algoritmo através de um experimento real, implementando dois algoritmos já estabelecidos, o algoritmo genético (Genetic Algorithm - GA) e o algoritmo de otimização de baleias (Whale optimization algorithm - WOA), e comparando a evolução de suas eficiências computacionais ao resolver um problema comum, sendo este o desafio do caixeiro viajante, desta forma, poderemos melhorar o entendimento desses algoritmos e a importância de sua implementação na resolução de um problema.

2 Experimento e algoritmos

Como descrito na sessão 1, o experimento será feito buscando uma solução para o Problema do Caixeiro Viajante (Travelling Salesman Problem - TSP). Segundo Larrañaga [1999], este problema é descrito como "Dado um número de cidades, determinar a menor rota que possamos visitar cada cidade exatamente uma vez antes de voltar para o ponto inicial", ou, de forma mais formal, "Dado um número $n \geq 3$ e uma matriz $n : n$. $C = (c_{ij})$, onde cada c_{ij} é um inteiro não negativo. Qual permutação cíclica π dos inteiros 1 a n minimiza a soma $\sum_i^n = 1^{c_{i\pi(i)}}$ ".

Ainda segundo Larrañaga [1999], Este representa um problema clássico de otimização frequentemente utilizado para problemas de otimização, por "Ser fácil de explicar, mas difícil de resolver", "Ser vastamente aplicável a uma variedade de problemas de roteamento e agendamento", e "por ser um problema bem conhecido, acaba servindo como um problema-teste para métodos de otimização". Segundo Talbi [2009], outro fator se dá pelo aumento explosivo no número de possíveis combinações e resultados conforme o nº de cidades aumenta, o que torna inviável alcançar a resposta por força bruta, isto é, avaliando todas as combinações individualmente.

Tabela 1. Efeito do nº de cidades na quantidade de combinações. Fonte: Talbi [2009]

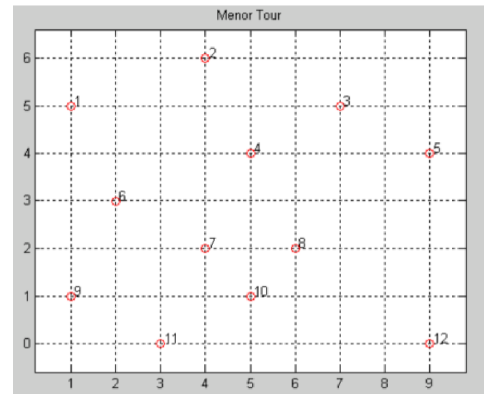
Nº de cidades n	Nº de combinações
5	120
10	3.628.800
75	$2.5 * 10^{109}$

Para efetivamente comparar a capacidade de otimização dos algoritmos escolhidos, foi gerada uma matriz com 12 cidades dispostas conforme a Figura 1, e se utilizou os algoritmos escolhidos com uma população de 50 e 100 em 200 gerações, avaliando a performance das soluções no decorrer de cada geração. Esse cenário prevê um total de 479.001.600 rotas possíveis.

2.1 Algoritmo genético

O Algoritmo Genético (Genetic Algorithm – GA) é uma técnica de otimização inspirada no processo de evolução natural dos seres vivos. Proposta por Holland [1975] A ideia principal do algoritmo é simular mecanismos semelhantes aos da sele-

Figura 1. Localização das cidades a serem visitadas



ção natural, reprodução e mutação para encontrar soluções cada vez melhores para um determinado problema.

No contexto do Problema do Caixeiro Viajante (Traveling Salesman Problem – TSP), cada solução possível representa uma rota que passa por todas as cidades exatamente uma vez e retorna à cidade inicial. Essas soluções são chamadas de indivíduos da população. Cada indivíduo possui uma aptidão (fitness), que indica a qualidade da solução. No TSP, geralmente a aptidão está relacionada à distância total percorrida, de forma que rotas menores representam soluções melhores.

O funcionamento do algoritmo genético ocorre de maneira iterativa. Inicialmente, é criada uma população com várias soluções aleatórias. Em seguida, cada solução é avaliada por meio de uma função de avaliação. Após essa etapa, os indivíduos mais aptos têm maior probabilidade de serem selecionados para gerar novos indivíduos.

A geração de novas soluções ocorre através de operadores genéticos. O operador de cruzamento (crossover) combina características de dois indivíduos para gerar descendentes. Já o operador de mutação realiza pequenas alterações aleatórias em uma solução, ajudando o algoritmo a explorar diferentes regiões do espaço de busca e evitando que ele fique preso em ótimos locais. Ao final de cada geração, uma nova população é formada e o processo se repete até que um critério de parada seja atingido, como um número máximo de gerações ou a estabilização da melhor solução encontrada. Neste experimento utilizamos os operadores OX (Order Crossover) e PMX (Partially Mapped Crossover), com a taxa de cruzamento (Cr) variando entre 0,8 e 0,9 e a taxa de mutação (Mt) variando entre 0,01 e 0,05.

Os algoritmos genéticos são amplamente utilizados em problemas de otimização combinatória devido à sua capacidade de explorar grandes espaços de busca e produzir boas soluções aproximadas em tempo viável, mesmo em problemas complexos como o TSP.

Pseudocódigo do Algoritmo Genético Abstrato: Fonte: Larrañaga [1999]

1. Faça uma população aleatória
2. Enquanto não parar, faça:
 - a Selecione parentes da população
 - b Produza crianças destes parentes
 - c Modifique esses indivíduos
 - d Extenda a população adicionando as crianças
 - e Reduza a população estendida

3. Retorne o melhor indivíduo encontrado

2.2 Algoritmo de otimização de Baleias

O Whale Optimization Algorithm (WOA) é um algoritmo de otimização inspirado no comportamento de caça das baleias jubarte (Humpback Whales). Esse algoritmo foi proposto por Mirjalili [2016] e faz parte da área de Inteligência de Enxame (Swarm Intelligence).

A principal inspiração do WOA é a estratégia de caça conhecida como bubble-net feeding, na qual as baleias criam trajetórias em espiral ao redor de suas presas para capturá-las. O algoritmo utiliza esse comportamento natural como base para realizar a busca por soluções ótimas em problemas de otimização.

No WOA, cada baleia representa uma possível solução do problema. No caso do Problema do Caixeiro Viajante, cada baleia pode representar uma rota diferente entre as cidades. Durante a execução do algoritmo, as baleias atualizam suas posições no espaço de busca tentando se aproximar das melhores soluções encontradas até o momento.

O algoritmo possui três fundamentos principais: Rodeamento da presa, Exploração e Exploração. A fase de rodeamento acontece quando uma baleia identifica a posição de uma presa e começa a se movimentar ao redor dela. Essa fase acontece após a identificação do melhor agente, e é descrita pelas seguintes equações:

$$\begin{aligned} \vec{D} &= |\vec{C} \cdot \vec{X}^*(t) - \vec{X}(t)| \\ \vec{X}(t+1) &= \vec{X}^*(t) - \vec{A} \cdot \vec{D} \end{aligned}$$

sendo que t indica a iteração atual, $\vec{A}$ e $\vec{C}$ são vetores de coeficiente, X^* é a melhor solução encontrada até aquele momento, $\vec{X}$ é o vetor de posição. Os vetores $\vec{A}$ e $\vec{C}$ são calculados com $\vec{A} = 2\vec{a} \cdot \vec{r} - \vec{a}$ e $\vec{C} = 2 \cdot \vec{r}$, em que $\vec{a}$ decai linearmente de 2 a 0 no decorrer das iterações, e $\vec{r}$ é um valor aleatório em $[0, 1]$

O comportamento de aproximação da presa representa a fase de exploração realizada através de bubble-net feeding e movimento de espiral ao redor da presa, essa fase auxilia no refinamento das soluções ao longo das iterações e é descrita como:

1. Mecanismo de encolhimento envolvente: Reduzindo o valor de $\vec{a}$, isso reduz a faixa de alcance de $\vec{A}$ e faz com que futuras posições de um agente podem estar em qualquer lugar entre sua posição atual e a posição do melhor agente, como demonstrado na Figura 2.
2. Atualização em espiral de posição: Pela figura 3, esse mecanismo calcula a distância entre uma baleia em (X, Y) e sua presa localizada em X^*, Y^* . Uma equação é criada para imitar o movimento em hélice: $\vec{X}(t+1) = \vec{D}' \cdot e^{bl} \cdot \cos(2\pi l) + \vec{X}^*(t)$ sendo $\vec{D}' = |\vec{X}^*(t) - \vec{X}(t)|$, representando a distância da i^a baleia à presa (a melhor solução), b é uma constante para definir o formato da espiral, l é um número aleatório em $[-1, 1]$.

As baleias nadam ao redor da presa utilizando o movimento circular que encolhe com o tempo e o movimento em espiral simultaneamente, para imitar esse comportamento, o algoritmo escolhe aleatoriamente qual desses mecanismos

ativar por vez com 50% de chance de cada, com o seguinte modelo matemático

$$\vec{X}(t+1) = \begin{cases} \vec{X}^*(t) - \vec{A} \cdot \vec{D} & : p < 0.5 \\ \vec{D}' \cdot e^{bl} \cdot \cos(2\pi l) + \vec{X}^*(t) & : p \geq 0.5 \end{cases}$$

A fase de exploração é responsável por buscar novas regiões do espaço de soluções, aumentando a diversidade e reduzindo a chance de convergência prematura, é feita de maneira oposta à fase de exploração, aumentando a faixa de $\vec{A}$ para forçar agentes a se moverem para mais longe das baleias de referência, além de escolher baleias aleatórias ao invés da baleia ótima, seguindo o modelo matemático:

$$\begin{aligned} \vec{D} &= |\vec{C} \cdot \vec{X}_{rand} - \vec{X}| \\ \vec{X}(t+1) &= \vec{X}_{rand} - \vec{A} \cdot \vec{D} \end{aligned}$$

sendo X_{rand} uma posição de uma baleia aleatória na população.

O algoritmo começa com um conjunto de soluções aleatórias. À cada iteração, os agentes se movimentam de acordo com a melhor solução obtida ou com alguma outra solução aleatória. O parâmetro a decresce de 2 a 0 para prover exploração ou exploração, respectivamente. Um agente aleatório é escolhido quando $|\vec{A}| > 1$ e o melhor solução é escolhida quando $|\vec{A}| < 1$ para a atualização da posição dos agentes. Dependendo do valor de p o algoritmo escolhe entre um movimento espiral ou circular, finalmente, o algoritmo termina ao satisfazer um critério de fim.

O WOA tem se destacado por apresentar boa capacidade de exploração e convergência, sendo aplicado em diversos problemas de otimização contínua e combinatória. Sua simplicidade de implementação e eficiência tornam o algoritmo uma alternativa interessante para a resolução do Problema do Caixeiro Viajante.

Pseudocódigo do Algoritmo de Otimização de Baleias, Fonte: Mirjalili [2016]:

- Inicie a população de baleias X_i ($i = 1, 2, 3, \dots, n$)
- Calcule a aptidão de cada agente de pesquisa
- X^* é o melhor agente de pesquisa
- enquanto ($t < \text{máximo de iterações}$):
 - Para cada agente: Atualize a , A , C , l e p
 - Se ($P < 0,5$):
 - Se ($|\vec{A}| < 1$):
 - Atualize a posição do agente atual: $\vec{D} = |\vec{C} \cdot \vec{X}^*(t) - \vec{X}(t)|$
 - Senão se ($|\vec{A}| \geq 1$):
 - Escolha um agente aleatório X_{rand}
 - Atualize a posição do agente para $\vec{X}(t+1) = \vec{X}_{rand}(t) - \vec{A} \cdot \vec{D}$
 - Senão se ($P \leq 0,5$):
 - Atualize a posição do agente atual para $\vec{X}(t+1) = \vec{D}' \cdot e^{bl} \cdot \cos(2\pi l) + \vec{X}^*(t)$
 - Cheque se algum agente saiu do campo de pesquisa e corrija.
 - calcule a aptidão de cada agente
 - atualize X^* se houver uma solução melhor
 - $t = t+1$
 - retorne X^*

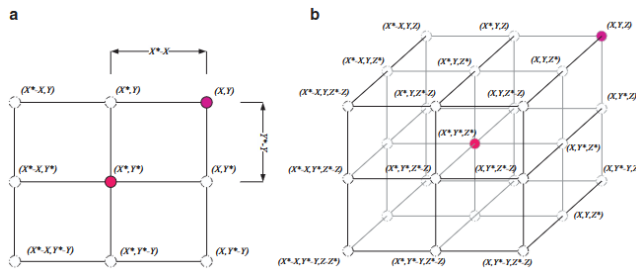


Figura 2. Vetores 2D e 3D e suas próximas localizações em relação a melhor posição. Fonte: Mirjalili [2016]

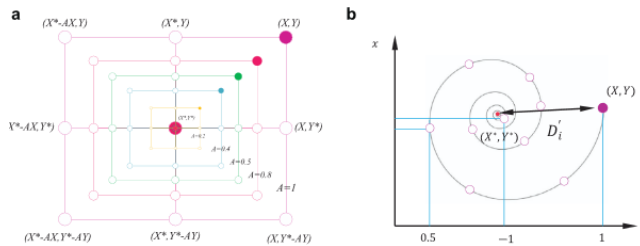


Figura 3. Mecanismo de busca e espiral de atualização de posição. Fonte: Mirjalili [2016]

3 Resultados

Foram realizados um total de 18 testes, sendo 16 com o algoritmo genético (AG) e 2 com o algoritmo de otimização de baleias (WOA), com seus parâmetros configurados de acordo com a tabela 2 e 3. O teste consistiu em solucionar o Problema do Caixeiro Viajante em um mesmo mapa com 12 cidades dispostas de acordo com a Figura 1, visando obter o melhor resultado, nesse contexto, sendo a menor rota entre as cidades (Ou seja, minimizando a aptidão), ao final do teste se extraiu o nível de aptidão mínima, máxima, média, mediana e calculando o desvio padrão, essas informações estão dispostas na tabela 4 e 5.

Tabela 2. Atributos dos testes de AG

ID	Pop	Cr	Mt	OP
AG_C1	50	0,8	0,01	OX
AG_C2	50	0,8	0,01	PMX
AG_C3	50	0,8	0,05	OX
AG_C4	50	0,8	0,05	PMX
AG_C5	50	0,9	0,01	OX
AG_C6	50	0,9	0,01	PMX
AG_C7	50	0,9	0,05	OX
AG_C8	50	0,9	0,05	PMX
AG_C9	100	0,8	0,01	OX
AG_C10	100	0,8	0,01	PMX
AG_C11	100	0,8	0,05	OX
AG_C12	100	0,8	0,05	PMX
AG_C13	100	0,9	0,01	OX
AG_C14	100	0,9	0,01	PMX
AG_C15	100	0,9	0,05	OX
AG_C16	100	0,9	0,05	PMX

Tabela 3. Atributos dos testes de WOA

ID	Pop
WOA_P50	50
WOA_P100	100

Tabela 4. Resultados do teste de AG

ID	Melhor	Pior	Média	Mediana	DP
AG_C1	29,25	33,41	30,15	29,25	1,50
AG_C2	29,25	35,24	30,92	31,26	1,77
AG_C3	29,25	32,10	29,39	29,25	6,38
AG_C4	29,25	33,99	30,30	29,25	1,60
AG_C5	29,25	34,62	30,50	29,25	1,71
AG_C6	29,25	36,16	31,32	30,54	2,31
AG_C7	29,25	32,31	29,40	29,25	6,84
AG_C8	29,25	33,69	30,28	29,25	1,55
AG_C9	29,25	33,18	29,75	29,25	1,11
AG_C10	29,25	34,67	31,19	31,51	1,64
AG_C11	29,25	29,25	29,25	29,25	0,00
AG_C12	29,25	32,94	30,42	29,25	1,41
AG_C13	29,25	32,31	30,20	29,25	1,34
AG_C14	29,25	32,94	30,21	29,25	1,41
AG_C15	29,25	29,25	29,25	29,25	0,00
AG_C16	29,25	32,10	29,92	29,25	9,62

Tabela 5. Resultados do teste de WOA

ID	Melhor	Pior	Média	Mediana	DP
WOA_P50	31,51	40,62	34,92	34,39	2,83
WOA_P100	29,25	41,66	33,93	33,51	3,33

A primeira vista, o algoritmo de otimização de baleias não obteve a mesma performance que o algoritmo genético, por não alcançar os níveis que algumas implementações da AG. Entretanto, é importante lembrar que o objetivo de algoritmos meta-heurísticos como estes não é buscar a solução ótima, mas sim, segundo Talbi [2009], prover soluções "aceitáveis" para problemas complexos em um tempo razoável. Nesse contexto verificamos a evolução da aptidão encontrada no decorrer das gerações, conforme demonstrado as figuras 4 e 5, mas, pelas tabelas pode-se verificar que o desvio padrão dos testes de AG varia muito mais que os testes de WOA, em particular AG_C7 e AG_C16 mostrando uma variação muito maior que os verificados na WOA. Isso indica que o AG depende de um minucioso controle de seus parâmetros, enquanto WOA é capaz de prover soluções equiparáveis sem ter de se preocupar com o controle fino de seu comportamento.

As figuras 4 e 5 demonstram a evolução da aptidão média ao longo das gerações. A partir delas, é possível verificar que a WOA encontra uma estagnação de forma bem comparável a algumas implementações da AG, em cerca de 75 a 100 gerações, enquanto outras implementações da AG permanecem se otimizando após 150.

Diante disso, a WOA se demonstra como uma boa solução para situações em que a minimização do tempo de implementação e testes seja prioridade, que é a principal promessa da família de algoritmos meta-heurísticos.

4 Conclusões e estudos relacionados

Uma questão recorrente no emprego de algoritmos meta-heurísticos é o balanceamento entre tentar alcançar a melhor solução possível e gastar o menor tempo possível. Segundo Dorigo [2004] Um problema recorrente dessa família de al-

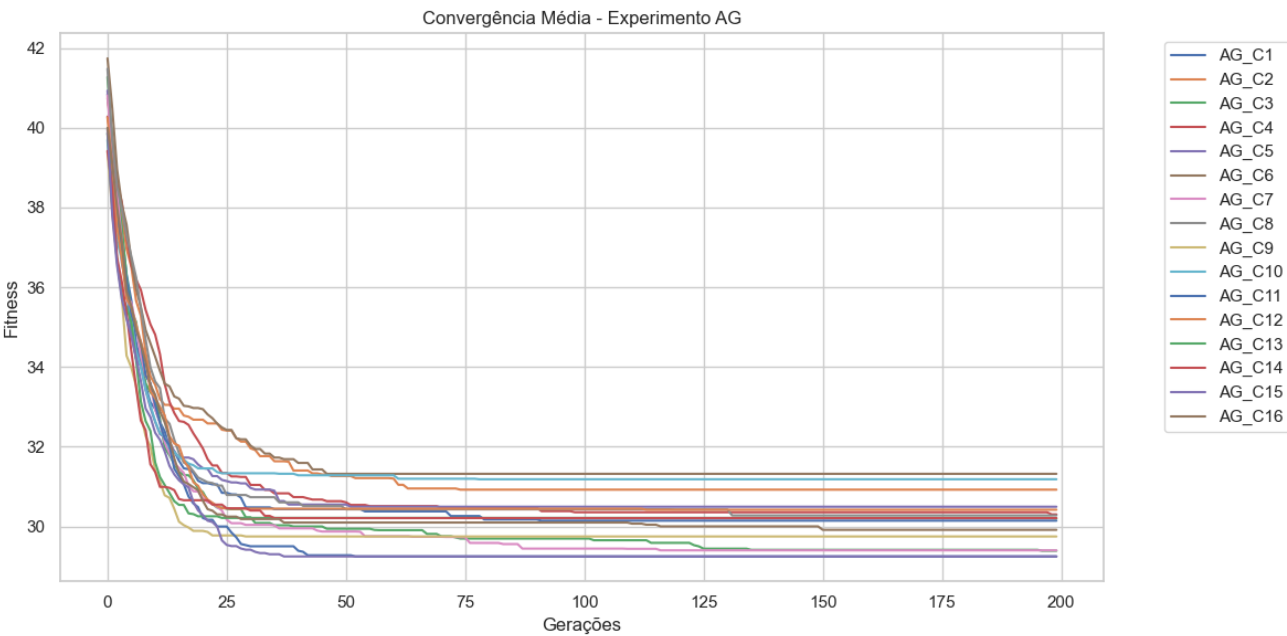


Figura 4. Curva de convergência dos testes de AG

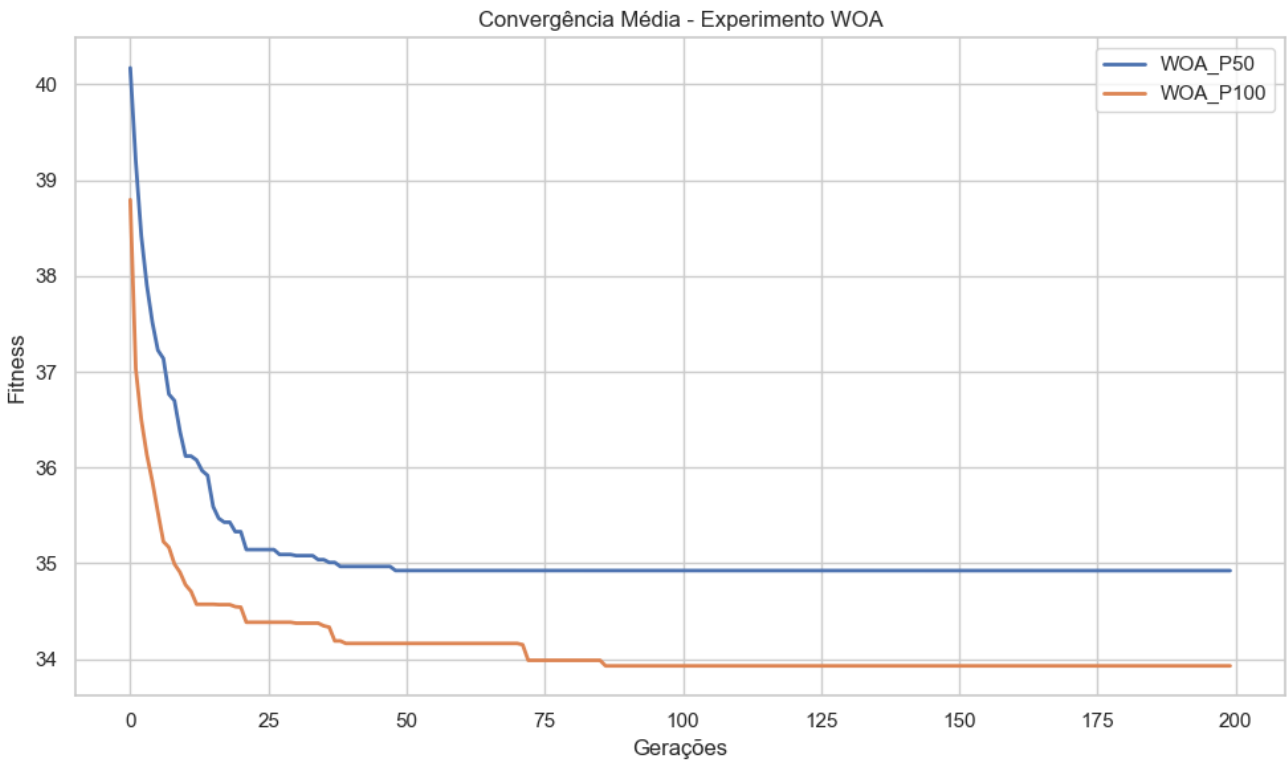


Figura 5. Curva de convergência dos testes de WOA

goritmos, presente inclusive no WOA, é a chamada "Convergência prematura", definida pela presunção de uma melhor solução local como a melhor solução para um problema, sem conseguir encontrar a solução ótima do problema como um todo. Além disso, Chakraborty [2020] diz que todos os algoritmos encontrados possuem suas limitações e que pesquisadores podem aliviar tais limitações utilizando outras meta-heurísticas ou componentes de meta-heurísticas, mas destaca que não há um algoritmo de otimização que seja generalista o bastante para ser a melhor opção para todos os problemas.

Chakraborty [2020] atribui as limitações do WOA à sua baixa capacidade de exploração, baixa velocidade de convergência e fica preso à uma solução facilmente, e traz vários estudos de algoritmos que embarcam o WOA e melhoram sua performance, tais como Levy flight trajectory based WOA (LWOA), Modified WOA (MWOA), Balanced WOA (BWOA), WOA with Quadratic interpolation (QIWOA), Lamarckian learning-based WOA (WOALam), Reinforced WOA (RWOA), Improved WOA (IWOA), Hybrid WOA Nelder-Mead local search algorithm (HWOANM), além de propor o WOA with a modified mutualism phase (WOAmM), que modifica e integra a fase de mutualismo da pesquisa de organismos simbióticos (Symbiotic Organisms Search - SOS) com a WOA, buscando reduzir a convergência prematura.

Essa amplitude de estudos e algoritmos desenvolvidos a partir de um único algoritmo demonstram como o uso e o desenvolvimento de um algoritmo meta-heurístico é, além de se tratar de uma mera busca de otimização, é um processo criativo e inovador, capaz de fornecer diversas formas distintas de se encarar um problema. Ao eliminar a necessidade de uma solução ótima, o programador passa a conseguir otimizar os demais aspectos de uma solução.

O escopo desse trabalho é extremamente estrito, com apenas 2 algoritmos meta-heurísticos sendo usados para solucionar apenas 1 problema em comum, sua função primordial foi prover uma experiência aos autores da utilização de algoritmos meta-heurísticos, como implementá-los para a solução de um problema, e compreender as vantagens inerentes dessa família de algoritmos na solução de problemas complexos.

Futuros trabalhos podem ser feitos explorando as variações citadas do WOA, como eles foram implementados, como eles diferem do original, a ideia que motivou eles, ou até mesmo buscar o desenvolvimento de uma variação própria.

Declarações complementares

Agradecimentos

Agradecemos ao professor Edmilson Batista dos Santos, mentor da disciplina de Inteligência Artificial da Universidade Federal de São João Del Rei, por proporcionar este trabalho para sua turma.

Contribuições dos autores

Todos os autores contribuíram para a concepção desse estudo. Gabriel Silva e Ícaro Cook implementaram os algoritmos escolhidos dentro do problema destacado. Fernando Alves e João Nascimento levantaram os materiais bibliográficos e contribuíram para esse manuscrito, todos os autores leram e aprovaram o manuscrito final.

Conflitos de interesse

Os autores declaram não possuir nenhum conflito de interesses.

Disponibilidade de dados e materiais

Os códigos desenvolvidos, bem como a matriz de cidades utilizada, estão disponíveis em https://github.com/LowEfficiency/tabalho_ja.git.

Referências

- Bastos, Tatiane Roldão; Longaray, A. A. (2025). Meta-heurísticas para a tomada de decisão multicritério: revisão sistemática da literatura e oportunidades de pesquisa. *Innovar*, 35(96):1–13. DOI: 10.15446/innovar.v35n96.104948.
- Chakraborty, Sanjoy; Saha, A. K. S. S. M. S. C. S. (2020). A novel enhanced whale optimization algorithm for global optimization. *Computers and Industrial Engineering*.
- Dorigo, M.; Stützle, T. (2004). *Ant Colony Optimization*. MIT Press.
- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- Larrañaga, P.; Kuijpers, C. M. H. M. R. H. I. I. D. S. (1999). Genetic algorithms for the travelling salesman problem: A review of representations and operators. *Artificial Intelligence Review*, 13:129–170.
- Luger, G. F. (2013). *Inteligência Artificial*. Pearson Education do Brasil, 6ª edition.
- Mirjalili, S.; Lewis, A. (2016). The whale optimization algorithm. *Advances in Engineering Software*, 95:51–67.
- Talbi, E.-g. (2009). *Metaheuristics: From Design to Implementation*. Wiley.

[H]